Homework 3 – Multicast Heartbeat Control (5 points)

Due: May 8, 11:30 PM

Submission Requirements

You need to submit a compressed archive of your homework directory. Under your homework directory, you should include:

- All your source code.
- A Makefile for compiling the program.
- A README file that describes how to compile and test your program.

Before submitting, create a gzipped tarball (.tar.gz) of your entire homework directory using the following command:

tar -czvf username-homework-3.tar.gz your_homework_directory/

where username is your NETID.

Submit the tarball on Canvas.

Objective:

This assignment requires you to implement a distributed system using UDP multicast where a controller can instruct multiple clients to start or stop sending heartbeat messages and adjust the heartbeat interval. The system should also include a mechanism for clients to detect if the controller fails and for one of the clients to take over as the new controller.

Language: C

No External Libraries: The solution should rely on standard C libraries for networking (e.g., `socket.h, netinet/in.h, arpa/inet.h, unistd.h, stdio.h, stdlib.h, string.h, time.h, sys/select.h or sys/poll.h`).

- This requirement ensures that students are learning and applying the fundamental networking concepts provided by the standard C libraries.
- Platform Independence (mostly)

Program Startup and Roles:

The program will be executed from the command line with the following syntax:

Bash

```
./your_program <user_id> [options]
```

- `<user_id>`: A unique identifier for this instance (max 32 characters).
- `-m <multicast_ip>` or `--multicast-ip=<multicast_ip>`: Override default multicast IP (239.0.0.1).

- `-p <port>` or `--port=<port>`: Override default port (12345).
- `-c <initial_interval>`: If present, this instance starts as the initial controller using <initial_interval> (integer in [0-30]) as the initial heartbeat interval. If absent, it starts as a regular client.

Every process starts as either a potential controller (if -c is used) or a client. A client can dynamically become a controller if no other controller is detected.

Network Communication:

All communication will be over UDP multicast using the specified multicast group IP address and port number.

Controller Functionality:

1. **User Interface:**
   - If started with the -c flag, the program will wait a pre-defined command interval (e.g., 5 seconds, which we will discuss later) for possible conflict with existing controller. If it receives any controller commands before timeout, it should log this command and automatically converts itself to a regular client and log a line in its log file: "<time-stamp> <user_id>: becomes client due to existing controller." If it didn't see any controller message before timeout, it starts acting as the controller, sending out SET_INTERVAL commands with the provided <initial_interval>.
   - The controller will use select() or poll() to monitor both standard input (stdin) and the multicast socket.
2. **Command Processing:**
   - User can interactively set new heartbeat interval value on terminal. When the user enters an integer between 0 and 30 (inclusive):
     - The controller will immediately send a new SET_INTERVAL command to the multicast group with the entered interval value.
     - The controller should log the valid user input and the SET_INTERVAL command sent. (Data structure is defined in Message Structure section.)
   - The controller should periodically re-send the last issued SET_INTERVAL command at a predefined command interval (e.g., 5 seconds).
   - The controller should log when it assumes the controller's role (if it started as a client and took over).
3. **Graceful Exit:**

   If the user enters "*exit*", the controller will send a CONTROLLER_DOWN command, log that it is relinquishing the controller role, optionally send a "BYE" message, close its log file, close its socket, and terminate. (Data structure is defined in Message Structure.)
4. **Abnormal Termination:**

   If the user enters "*abort*", the controller will close its log file and call abort().

5. **Invalid Input:**

   If the user enters invalid input for the heartbeat interval, the controller should display an error message on the terminal but not log the invalid input.

6. **Log** all received multicast messages except those sent by itself (see details in Logging Requirements)

Client Functionality:

1. **Joining Multicast Group:** Clients will join the specified multicast group.
2. **Heartbeat Timer:** Clients will have a programmable timer for sending heartbeat messages containing their user_id. (Data structure defined in Message Structure.)
3. **Command Reception:** Clients will listen for SET_INTERVAL and CONTROLLER_DOWN commands.
4. **Heartbeat Control:** Clients will start, stop, and adjust their heartbeat sending based on valid SET_INTERVAL commands received from the controller.
   - Stopping Heartbeats: If the received interval value is 0, clients will cease sending heartbeat messages.
   - Starting/Adjusting Heartbeats: If the received interval value is between 1 and 30 (inclusive, representing seconds), clients will begin (or adjust) periodically sending heartbeat messages. The first heartbeat will be sent after a random initial delay of 0.0 to 0.9 seconds (inclusive, with millisecond precision), calculated based on the client's PID. Implementing a random jitter of up to +/- 10% on each subsequent heartbeat interval is an optional enhancement.
   - Each subsequent heartbeat will be sent with the specified interval with optional random jitter of up to 10% of that interval, to prevent long-term synchronization.".
5. Client Heartbeat - **Randomized Initial Delay**:

   When a client receives a SET_INTERVAL command with a new non-zero interval (different from previous one), it must implement a random initial delay before starting its periodic heartbeat transmissions (to avoid client collision). This delay should be a random floating-point number within the range of 0.0 to 0.9 seconds (inclusive), with uniform distribution and millisecond precision. The random number generator should be seeded using the client's Process ID (PID) to help ensure unique delays across different client instances. After this initial delay, the client will begin sending heartbeats at the specified interval, optionally with added jitter.
6. *Optional* **jitter** (no bonus): After the initial random delay, when a client starts sending heartbeats at the interval specified by the SET_INTERVAL command, it must introduce a small amount of random jitter to each subsequent heartbeat interval. This jitter, randomly chosen within a range of plus or minus 10% of the interval, will ensure that clients do not maintain a perfectly periodic heartbeat transmission pattern and thus avoid potential long-term synchronization that could lead to periodic network congestion.

   For example, if a client receives a SET_INTERVAL of 5 seconds:

   1) It waits for its PID-based random initial delay (e.g., 0.3 seconds).

2) It sends the first heartbeat at 0.3 seconds.
3) If no jitter, it sends the next heartbeat 5 seconds later, and so on. If you choose to implement optional jitter, go to 4) and 5).
4) For the next heartbeat, it calculates a jitter value (e.g., -0.4 seconds, which is within +/- 10% of 5 seconds). The next heartbeat will be sent at approximately 5 + (-0.4) = 4.6 seconds after the previous one.
5) For the subsequent heartbeat, a new random jitter value will be generated and applied to the base interval, and so on.

7. **Missing Controller Detection:** Clients will detect a missing controller after a predefined number of missed periodic SET_INTERVAL commands (e.g., 3 intervals) or upon receiving a CONTROLLER_DOWN command.
8. **Controller Takeover:** Upon detecting a missing controller, the client will prompt the user with: "Become the new controller? Enter heartbeat interval (0-30):".
   - If the user enters an integer between 0 and 30, the client becomes the new controller, using the entered value as the initial interval. If it has not received any new controller command from other nodes yet (see "Handling Simultaneous Takeover"), it starts sending `SET_INTERVAL` commands. This action should be logged.
   - Invalid input should result in an error message on the terminal, and the client remains a regular client (no log entry for the invalid attempt), and wait for valid input.
   - When a client detects a new controller command from another node, it turns off its takeover mode and returns to normal client mode. Any user input afterwards will be rejected as invalid input (and not be logged), until it detects missing controller (see Missing Controller Detection.)
9. **Handling Simultaneous Takeover:** A client that becomes a potential new controller will first check whether there has been any new controller command from the multicast socket before sending its own controller command.
   - If the answer is yes, the clients won't send their own controller commands, and reverts to normal client mode, and logging and printing a notification ("New controller detected at [IP Address]:[Port]. Returning to client mode.").
   - If the answer is no, it starts sending its controller commands. But it will keep listening to any potential new controller commands from other nodes for one command interval (e.g., 5 seconds) or less. If there is no new command from other nodes, it continues running as the controller.
   - If it receives controller commands from other nodes during that period (due to simultaneous takeover and network delay, though the event is extremely improbable to happen on your local host), it stops sending new commands and return to takeover mode. It logs and prints a notification ("New controller detected at [IP Address]:[Port] – conflict. Retry") and then prompts the user to retry.
   - Normal clients (other than the potential controllers) don't need special action to handle simultaneous takeover.
10. **Graceful Exit:** If the user enters "exit", the client will stop sending heartbeats, optionally send a "BYE" message, close its log file, close its socket, and terminate.

11. **Abnormal Termination:** If the user enters "abort", the client will close its log file and call abort().
12. **Unexpected Input:** During normal operation (when a controller is active), clients should reject any user input (other than "exit") on their terminal and display a message indicating that input is not expected. This invalid input should not be logged.

Example of Message Structures:

```
struct heartbeat_message {
    char user_id[32];
};

struct command_message {
    char command_type[16]; // Always "SET_INTERVAL"
    int interval;          // 0 for off, 1-60 for interval in seconds
};

struct controller_down_message {
    char command_type[16]; // Always "CONTROLLER_DOWN"
};

struct bye_message {
    char user_id[32];
};
```

You can design your own message structures.

Timer Implementation Hint:

For implementing both the initial random delay and the periodic heartbeat transmissions (and for the controller's command re-sends), consider using the select() or poll() system calls with appropriate timeout values. These functions not only allow you to wait for activity on file descriptors (like your multicast socket and standard input) but can also serve as high-resolution timers (down to microseconds for select() and milliseconds for poll()) when called with a timeout and no file descriptors to monitor.

Crucially, if select() or poll() returns due to an event occurring before the scheduled timeout for your periodic task, you will need to recalculate the remaining time until that task should have executed. To do this:

1. **Record the intended time** for the next execution of your periodic task (e.g., next heartbeat).
2. **When select() or poll() returns early, handle the received message(s) or other events.**
3. **After handling the events, get the current time**.
4. **Calculate the difference between the intended execution time and the current time**. This difference is the remaining time until your periodic task should run.

5. **Call select() or poll() again** with this remaining time as the timeout, along with your file descriptors.

This ensures that your periodic tasks stay on schedule even when other events are being handled asynchronously. You will need to carefully manage time using functions like gettimeofday() (for select()) or by tracking elapsed time with poll() to implement this correctly.

Logging Requirements:

❖ Log **file name**: `heartbeat_app_<user_id>.log`, opened in append mode (`"a"`).

Whenever a controller or a client receives a UDP packet from the multicast socket that **did not originate from itself**, they should log the following information:

- **Timestamp:** The exact time the message was received.
- **Source IP Address and Port:** The IP address and port number of the sender of the message. (This is crucial for distinguishing between messages from the controller and from other clients, especially during a controller takeover scenario.)
- **Sender ID:**
  - For a received SET_INTERVAL or CONTROLLER_DOWN command, log "controller" as the sender (because the commands have no sender info).
  - For a received heartbeat message, log the user_id from the heartbeat_message.
  - For a received BYE message, log the user_id from the bye_message.
  - For uninterpretable data, the sender ID might not be available.
- **Received Data (as interpreted):**
  - If the received data is interpreted as a command (`SET_INTERVAL` or `CONTROLLER_DOWN`), log the command type and its value (if applicable).
  - If the received data is interpreted as a heartbeat message, log that it's a heartbeat and include the client identifier.
  - If the received data is interpreted as a "BYE" message, log that it's a BYE message and include the identifier and timestamp from the message.
  - If the received data cannot be interpreted according to your defined message formats, log the raw bytes received (perhaps in hexadecimal format) and a warning indicating an uninterpretable message.

Additionally, each node should have logic to typically ignore processing messages it sent itself, even though it will likely receive them due to multicast loopback.

- Log the **sending of all multicast messages** (commands, heartbeats, BYE) by the **sender**.
- Log **valid user commands** like **"exit**" or **"abort**"
- *Controller* logs all **valid user inputs for heartbeat interval.**
- Log **when a node assumes** or **relinquishes** the **controller role**.

Log format:

[YYYY-MM-DD HH:MM:SS.milliseconds] user_id: Your log message here

Example 1

[2025-04-25 22:57:58.349] userB: No controller detected. Initiating takeover with interval 10 and assuming controller role.
[2025-04-26 22:57:58.351] userB: Sending command: SET_INTERVAL 10
[2025-04-25 22:58:00.162] userB: Received Heartbeat from userX (192.168.1.101:6805)
[2025-04-25 22:58:01.456] userB: Received Heartbeat from userC (192.168.1.101:6001)
…
[2025-04-25 23:02:58.694] userB: Sending command: SET_INTERVAL 10
…
[2025-04-26 00:06:15.456] userB: User entered interval: 0
[2025-04-26 00:06:15.458] userB: Sending command: SET_INTERVAL 0
…
[2025-04-26 00:12:56.789] userB: User entered 'exit'. Send CONTROLLER_DOWN.
[2025-04-26 00:12:56.792] userB: Node userB releases controller role.
[2025-04-26 00:12:56.795] userB: Sending BYE

Example 2

[2025-04-26 00:05:12.385] userC: Received command: SET_INTERVAL 10 from controller (192.168.1.100:5000)
[2025-04-25 22:58:00.163] userC: Received Heartbeat from userX (192.168.1.101:6805)
[2025-04-25 22:58:01.438] userC: Sending Heartbeat
…
[2025-04-25 23:02:58.703] userC: Received command: SET_INTERVAL 10 from controller (192.168.1.100:5000)
…
[2025-04-26 00:06:15.468] userC: Received command: SET_INTERVAL 0 from controller (192.168.1.100:5000)
… *(stop sending heartbeat until receiving new positive heartbeat interval.)*
[2025-04-26 00:12:56.794] userC: Received signal CONTROLLER_DOWN from controller (192.168.1.100:5000)
[2025-04-26 00:12:56.802] userC: Received BYE from userB (192.168.1.100:5000)

In your logs, probably all IPs will be 127.0.0.1 because you run all nodes on the same host.

Logging Utility Functions:

To promote consistent log formatting, which will greatly aid in grading, we strongly recommend that you use the provided logging utility files: logger.h and logger.c. These files offer a convenient way to generate logs in the required format. You can download the logging_utils.zip file containing these utilities along with the assignment.

Using the Provided Utilities (Recommended):

1. Download the logging_utils.zip file provided with the assignment. This zip file contains logger.h and logger.c.
2. Include logger.h in any of your .c files where you need to perform logging:

```
#include "logger.h"
```

3. Compile logger.c along with your main program and any other source files. For example, if your main file is heartbeat_app.c, your compilation command might look something like:

```
gcc heartbeat_app.c logger.c -o heartbeat_app
```

4. Initialize the logger in your main() function by calling init_logger(). This function takes your user_id (obtained from the command line) as an argument and returns a file pointer (FILE *) for your log file. Make sure to handle potential errors if the log file cannot be opened.

```
FILE *log_file = init_logger(user_id);
if (log_file == NULL) {
    // Handle error, perhaps exit the program
}
```

5. Use the log_message() function to write log entries. This function takes the log file pointer (log_file), your user_id, a format string (similar to printf), and any subsequent arguments to be formatted.

```
log_message(log_file, user_id, "User entered
interval: %d\n", interval);
log_message(log_file, user_id, "Received heartbeat
from %s (%s:%d)", sender_id, sender_ip, sender_port);
```

6. Close the log file before your program exits by calling close_logger():

```
close_logger(log_file);
```

Flexibility and Extensions:

While we strongly encourage the use of these utilities to ensure the base log format is correct, you are permitted to modify, rewrite, or extend the provided code to better suit your program's structure. For example, you might choose to create wrapper functions around log_message() to log specific events with more semantic names (e.g., log_sent_set_interval(), log_received_heartbeat()).

Crucially, any modifications or extensions you make must ensure that the final log output adheres to the following format for all log entries:

[YYYY-MM-DD HH:MM:SS.milliseconds] user_id: Your log message here

Failure to maintain this core log format may result in grading penalties.

Error Handling:

As usual, implement basic error handling for socket operations, network-related functions, and logging.